Universidad Nacional Autónoma de México

Ingeniería en ...

Nombre de Asignatura

FORMATO DE REPORTE

ALUMNO:
309075318

Grupo:
#
Semestre:
2024-II

México,CDMX. Febrero 2024

Índice

1. Introducción **2**

2. Marco Teórico **2**

3. Desarrollo **2**

3.1. Algoritmo: Carrito de Compra con Estructura De Datos 2

3.1.1. Descripción de Funciones 3

3.2. Pruebas 5

3.2.1. Pruebas de Inserción 5

3.2.2. Pruebas de Eliminación 5

3.2.3. Pruebas de Impresión 6

3.2.4. Pruebas de Impresión 6

4. Resultados **6**

5. Conclusiones **6**

1. Introducción

Este apartado debe abordar los siguientes puntos:

- **Planteamiento del problema.** Se hace una descripción del problema a resolver.
- **Motivación.** Se describe porqué es necesario dar solución al problema.
- **Objetivos.** Lo que se se espera obtener de darle solución al problema.

2. Marco Teórico

Son los conceptos aplicados, principalmente los vistos en clase, con los que se va a dar solución al problema, explicados de forma concreta, ya sea con su interpretación personal o bien con referencias bibliográficas sólidas. [1]

3. Desarrollo

3.1. Algoritmo: Carrito de Compra con Estructura De Datos

1. Inicializar la lista de productos vacía.
2. Mostrar el menú de opciones al usuario.
3. Leer la opción seleccionada.
4. Según la opción elegida, realizar una de las siguientes acciones:
   a) Agregar un nuevo producto a la lista.
   b) Buscar un producto por su ID.
   c) Eliminar un producto de la lista.
   d) Mostrar el producto actual.
   e) Avanzar al siguiente producto.
   f) Retroceder al producto anterior.
   g) Mostrar todos los productos en orden hacia adelante.
   h) Mostrar todos los productos en orden hacia atrás.
   i) Salir del programa.
5. Se libera la memoria utilizada.
6. Se finaliza el programa mostrando el ticket final.

3.1.1. Descripción de Funciones

- Función **crearNodo**: La función recibe tres argumentos: un entero **id**, una cadena **nombre** y un dato de tipo float **precio**. Su funcionamiento se divide en los siguientes pasos:

  1. Se reserva memoria dinámicamente para un nuevo nodo.
  2. Se verifica si la memoria fue asignada correctamente.
  3. Se asignan los valores de **id**, **nombre** y **precio** al nodo.
  4. Se inicializan los apuntadores **anterior** y **siguiente** en **NULL**.
  5. Se regresa el apuntador al nodo creado.

- Función **agregarProducto**: La función recibe el apuntador a la lista y los datos del producto. Su funcionamiento se divide en los siguientes pasos:

  1. Se crea un nuevo nodo.
  2. Se verifica si el nodo fue creado correctamente.
  3. Si la lista está vacía, el nodo se enlaza a sí mismo y se convierte en la cabeza.
  4. Si la lista no está vacía, se obtiene el último nodo.
  5. Se enlaza el nuevo nodo al final de la lista.
  6. Se actualizan los enlaces para mantener la estructura circular.

- Función **buscarProducto**: La función recibe la lista y un **id**. Su funcionamiento se divide en los siguientes pasos:

  1. Se verifica si la lista está vacía.
  2. Se recorre la lista con un apuntador auxiliar.
  3. Se compara cada **id** con el valor buscado.
  4. Si se encuentra coincidencia, se regresa el nodo.
  5. Si no se encuentra, se regresa **NULL**.

- Función **eliminarProducto**: La función recibe la lista y un **id**. Su funcionamiento se divide en los siguientes pasos:

  1. Se verifica si la lista está vacía.
  2. Se busca el nodo a eliminar.
  3. Si no existe, la función termina.
  4. Si solo hay un nodo, se elimina y la lista queda vacía.
  5. Si hay más nodos, se actualizan los enlaces de los nodos vecinos.
  6. Si el nodo eliminado es la cabeza, se actualiza al siguiente nodo.

7. Se libera la memoria del nodo eliminado.

- Función **mostrarProductoActual**: La función recibe un nodo actual. Su funcionamiento se divide en los siguientes pasos:

  1. Se verifica si el nodo es **NULL**.
  2. Si no lo es, se muestran los datos del producto.

- Función **siguienteProducto**: La función recibe un nodo actual. Su funcionamiento se divide en los siguientes pasos:

  1. Se verifica que el nodo no sea **NULL**.
  2. Se avanza al siguiente nodo.
  3. Se muestra el producto actual.

- Función **anteriorProducto**: La función recibe un nodo actual. Su funcionamiento se divide en los siguientes pasos:

  1. Se verifica que el nodo no sea **NULL**.
  2. Se retrocede al nodo anterior.
  3. Se muestra el producto actual.

- Función **mostrarProductos**: La función recibe la lista. Su funcionamiento se divide en los siguientes pasos:

  1. Se verifica si la lista está vacía.
  2. Se inicia el recorrido desde el primer nodo.
  3. Se muestra el producto actual.
  4. Se presenta un menú de navegación.
  5. Se avanza o retrocede según la opción elegida.
  6. Si el usuario decide salir, la función termina.

- Función **mostrarTicket**: La función recibe la lista. Su funcionamiento se divide en los siguientes pasos:

  1. Se verifica si la lista está vacía.
  2. Se recorre la lista desde el primer nodo.
  3. Se imprimen los datos de cada producto.

- Función **mostrarTicketAdelante**: La función recibe la lista. Su funcionamiento se divide en los siguientes pasos:

  1. Se verifica si la lista está vacía.
  2. Se recorre la lista en orden hacia adelante.

3. Se imprimen los datos de cada producto.

- Función **mostrarTicketAtras**: La función recibe la lista. Su funcionamiento se divide en los siguientes pasos:

  1. Se verifica si la lista está vacía.
  2. Se inicia desde el último nodo.
  3. Se recorre la lista hacia atrás.
  4. Se imprimen los datos de cada producto.

- Función **liberarLista**: La función recibe la lista. Su funcionamiento se divide en los siguientes pasos:

  1. Se verifica si la lista está vacía.
  2. Se recorren los nodos liberando la memoria de cada uno.
  3. Se libera el nodo inicial.
  4. Se asigna **NULL** a la lista.

- Función **mostrarMenu**: Le muestra al usuario las opciones del programa.

3.2. Pruebas

3.2.1. Pruebas de Inserción

Caso	ID	Nombre	Precio
1	101	Coca-Cola	18.50
2	102	Sabritas	15.00
3	103	Galletas	12.00
4	104	Agua	10.00

Tabla 1: Pruebas de inserción en la lista doble circular

3.2.2. Pruebas de Eliminación

Caso	ID eliminado	Resultado
1	103	Eliminado correctamente
2	200	No existe en la lista

Tabla 2: Pruebas de eliminación de productos

3.2.3. Pruebas de Impresión

Caso	Tipo de impresión	Resultado
1	Adelante	Lista en orden de inserción
2	Atrás	Lista en orden inverso

Tabla 3: Pruebas de impresión de productos

3.2.4. Pruebas de Impresión

Caso	Tipo de impresión	Resultado
1	Adelante	Lista en orden de inserción
2	Atrás	Lista en orden inverso

Tabla 4: Pruebas de impresión de productos

4. Resultados

Mediante capturas de pantalla y una breve descripción seguida de la captura se presentan los resultados finales de su aplicación.

5. Conclusiones

Se presenta un análisis de los resultados obtenidos, donde se destaca la importancia de la aplicación de los conceptos teóricos para resolver el problema. Es importante resaltar lo siguiente:

- No es describir si les gustó la actividad o no.
- No es decir que se obtuvo de la práctica.
- No es describir lo que fue difícil.

Referencias

[1] Brian W. Kernighan y Dennis M. Ritchie. *The C Programming Language*. 2nd. Prentice Hall Professional Technical Reference, 1988. ISBN: 0131103709.